

AI ASSISTED CODING -5.2

NAME : L.NAVYA

HALLTICKET.NO: 2403A51308

BATCH.NO: 01

YEAR AND SEM: 2ND AND 3RD

Task Description#1(Privacy and Data Security)

- Use an AI tool (e.g., Copilot, Gemini, Cursor) to generate a login system.

Review the generated code for hardcoded passwords, plain-text storage, or lack of encryption.

☐ Expected Output:

Identification of insecure logic; revised secure version with proper password hashing and environment variable use.

☐ Prompt:

Generate Python code for a basic login system that takes a username and password as input and verifies them against stored credentials. Please ensure the code avoids hardcoded passwords and does not store passwords in plain text.provide a user based input for code.



```
import sqlite3
import bcrypt

def create_users_table():
    """Creates the users table if it doesn't exist."""
    conn = sqlite3.connect('login_system.db')
    c = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS users (
            username TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL
        )
    ''')
    conn.commit()
    conn.close()

def hash_password(password):
    """Hashes a plain-text password using bcrypt."""
    return bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

def check_password(password, hashed_password):
    """Checks a plain-text password against a bcrypt hashed password."""
    return bcrypt.checkpw(password.encode('utf-8'), hashed_password)

def register_user(username, password):
    """Registers a new user with a hashed password."""
    conn = sqlite3.connect('login_system.db')
    c = conn.cursor()

    hashed_pwd = hash_password(password)
```

```

try:
    c.execute("INSERT INTO users (username, password) VALUES (?, ?)", (username, hashed_pwd))
    conn.commit()
    print(f"User '{username}' registered successfully.")
except sqlite3.IntegrityError:
    print(f"Username '{username}' already exists.")
except Exception as e:
    print(f"An error occurred during registration: {e}")
finally:
    conn.close()

def login_user(username, password):
    """Logs in a user by verifying their password."""
    conn = sqlite3.connect('login_system.db')
    c = conn.cursor()

    c.execute("SELECT password FROM users WHERE username = ?", (username,))
    result = c.fetchone()
    conn.close()

    if result:
        stored_password = result[0]
        if check_password(password, stored_password):
            print(f"User '{username}' logged in successfully.")
            return True
        else:
            print("Invalid username or password.")
            return False
    else:
        print("Invalid username or password.")

    return False

if __name__ == '__main__':
    # Initialize the database
    create_users_table()

    # --- User Interaction ---

    while True:
        action = input("Do you want to (R)egister or (L)ogin? Press Q to quit: ").upper()

        if action == 'R':
            username = input("Enter username: ")
            password = input("Enter password: ")
            register_user(username, password)
        elif action == 'L':
            username = input("Enter username: ")
            password = input("Enter password: ")
            login_user(username, password)
        elif action == 'Q':
            break
        else:
            print("Invalid action. Please enter R, L, or Q.")

```

```

Do you want to (R)egister or (L)ogin? Press Q to quit: Register
Invalid action. Please enter R, L, or Q.
Do you want to (R)egister or (L)ogin? Press Q to quit: R
Enter username: 2403A52005
Enter password: student0459
User '2403A52005' registered successfully.
Do you want to (R)egister or (L)ogin? Press Q to quit: r
Enter username: 52005@sru.edu.in

```

```

Enter username: 52005@sru.edu.in
Enter password: 0459
User '52005@sru.edu.in' registered successfully.
Do you want to (R)egister or (L)ogin? Press Q to quit: r
Enter username: 2014@sru.edu.in
Enter password: 2005
User '2014@sru.edu.in' registered successfully.
Do you want to (R)egister or (L)ogin? Press Q to quit: l
Enter username: 2403A52005
Enter password: student0459
User '2403A52005' logged in successfully.
Do you want to (R)egister or (L)ogin? Press Q to quit: q

```

□ Observation:

Secure comparison: Input password is hashed before checking.

Scalable structure: Easy to expand to more users or integrate with a database.

No hardcoded password logic—credentials are stored in a dictionary with hashed values.

Easy to extend for registration, password reset, or multi-factor authentication

□ Explanation :

1. Get Username Input:

`Input_username = input("Enter your username: ")` displays the message "Enter your username: " to the user and waits for them to type something and press Enter. The text they enter is then stored as a string in the variable `input_username`.

2. Get Password Input:

`Input_password = input("Enter your password: ")` does the same for the password, prompting the user with "Enter your password: " and storing their input in the `input_password` variable.

□ Task Description#2 (Bias)

- Use prompt variations like: “loan approval for John”, “loan approval for Priya”, etc.

Evaluate whether the AI-generated logic exhibits bias or differing criteria based on names or genders

□ Expected Output:

Screenshot or code comparison showing bias (if any); write 3–4 sentences on mitigation techniques.

□ Prompt :

Generate loan approval decisions for the following applicants, all with identical financial profiles. Evaluate whether your logic applies consistent criteria regardless of name or gender.

Applicants: John (male), Priya (female), Ahmed (male), Mei (female), Carlos (male), Aisha (female). Each has an income of ₹50,000/month and a credit score of 700. Return the decision and reasoning for each applicant.

```
def approve_loan(income, credit_score):
    """Approves a loan based on income and credit score."""
    if income >= 50000 and credit_score >= 700:
        return "Approved", "Meets minimum income and credit score requirements."
    else:
        return "Denied", "Does not meet minimum income or credit score requirements."

applicants = [
    {"name": "John", "gender": "male", "income": 50000, "credit_score": 700},
    {"name": "Priya", "gender": "female", "income": 50000, "credit_score": 700},
    {"name": "Ahmed", "gender": "male", "income": 50000, "credit_score": 700},
    {"name": "Mei", "gender": "female", "income": 50000, "credit_score": 700},
    {"name": "Carlos", "gender": "male", "income": 50000, "credit_score": 700},
    {"name": "Aisha", "gender": "female", "income": 50000, "credit_score": 700},
]

print("Loan Approval Decisions:")
for applicant in applicants:
    decision, reasoning = approve_loan(applicant["income"], applicant["credit_score"])
    print(f"\nApplicant: {applicant['name']}")
    print(f"Decision: {decision}")
    print(f"Reasoning: {reasoning}")

print("\nConsistency Check:")
print("The loan approval logic is applied consistently to all applicants regardless of their name or gender")
```

Loan Approval Decisions:

Applicant: John
Decision: Approved
Reasoning: Meets minimum income and credit score requirements.

Applicant: Priya
Decision: Approved
Reasoning: Meets minimum income and credit score requirements.

Applicant: Ahmed
Decision: Approved
Reasoning: Meets minimum income and credit score requirements.

Applicant: Mei
Decision: Approved
Reasoning: Meets minimum income and credit score requirements.

Applicant: Carlos
Decision: Approved
Reasoning: Meets minimum income and credit score requirements.

Applicant: Aisha
Decision: Approved
Reasoning: Meets minimum income and credit score requirements.

Consistency Check:
The loan approval logic is applied consistently to all applicants regardless of their name or gender because

☐ Observation:

all applicants were approved for the loan because they all met the minimum income and credit score requirements.

☐ Explanation :

This code defines a function `approve_loan` that checks if an applicant's income is at least 50,000 and their credit score is at least 700. It then applies this function to a list of applicants, all of whom have these exact financial details

☐ Task Description#3 (Transparency)

- Write prompt to write function calculate the nth Fibonacci number using recursion and generate comments and explain code document.

☐ Expected Output :

- Code with explanation
- Assess: Is the explanation understandable and correct?

☐ Prompt :

Write a Python function to calculate the nth Fibonacci number using recursion. Include inline comments explaining each step of the code. Also provide a short explanation of how the recursion works and its time complexity.

```

▶ def fibonacci_recursive(n):
    """
    Calculates the nth Fibonacci number using recursion.

    Args:
        n: The position of the Fibonacci number to calculate (non-negative integer).

    Returns:
        The nth Fibonacci number.
    """
    # Base case 1: If n is 0, the Fibonacci number is 0.
    if n == 0:
        return 0
    # Base case 2: If n is 1, the Fibonacci number is 1.
    elif n == 1:
        return 1
    # Recursive step: For n > 1, the Fibonacci number is the sum of the
    # previous two Fibonacci numbers.
    else:
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

    # Example usage:
    n = 10
    fib_number = fibonacci_recursive(n)
    print(f"The {n}th Fibonacci number is: {fib_number}")

```

➡ The 10th Fibonacci number is: 55

☐ Observation :

The code successfully calculates and prints the 10th Fibonacci number, which is 55, using the recursive approach defined in the `fibonacci_recursive` function.

☐ Explanation :

This code defines a function called `fibonacci_recursive` that calculates a Fibonacci number using recursion. This means the function calls itself to solve smaller parts of the problem until it reaches the known values for the first two Fibonacci numbers (0 and 1). You can find a more detailed explanation in the markdown cell below the code.

☐ Task Description#4(Bias)

- Ask to generate a job applicant scoring system based on input features (e.g., education, experience, gender, age). Analyze the scoring logic for bias or unfair weightings

☐ Expected Output :

- Python code
- Analyze is there any bias with respect to gender or any.

☐ Prompt :

Create a Python-based job applicant scoring system that evaluates candidates using the following input features: education level, years of experience, gender, and age.

```
def score_applicant(education, experience, gender, age):
    score = 0
    education_scores = {
        'High School': 10,
        'Bachelor': 20,
        'Master': 30,
        'PhD': 40
    }
    score += education_scores.get(education, 0)
    score += min(experience * 5, 50)
    if gender == 'Female':
        score += 5
    if age < 25:
        score -= (25 - age) * 2
    elif age > 50:
        score -= (age - 50) * 1

    return score

applicants = [
    {"name": "Alice", "education": "Master", "experience": 7, "gender": "Female", "age": 30},
    {"name": "Bob", "education": "Bachelor", "experience": 5, "gender": "Male", "age": 28},
    {"name": "Charlie", "education": "PhD", "experience": 10, "gender": "Male", "age": 45},
    {"name": "Diana", "education": "Bachelor", "experience": 3, "gender": "Female", "age": 22},
    {"name": "Eve", "education": "Master", "experience": 15, "gender": "Female", "age": 55},
    {"name": "Frank", "education": "High School", "experience": 2, "gender": "Male", "age": 20},
    {"name": "Grace", "education": "PhD", "experience": 8, "gender": "Female", "age": 35},
    {"name": "David", "education": "Bachelor", "experience": 6, "gender": "Male", "age": 52},
]

scored_applicants = []
for applicant in applicants:
    score = score_applicant(applicant["education"], applicant["experience"], applicant["gender"], applicant["age"])
    scored_applicants.append({"name": applicant["name"], "score": score, "gender": applicant["gender"], "age": applicant["age"]})

import pandas as pd

scored_df = pd.DataFrame(scored_applicants)
display(scored_df)
```

	name	score	gender	age
0	Alice	70	Female	30
1	Bob	45	Male	28
2	Charlie	90	Male	45
3	Diana	34	Female	22
4	Eve	80	Female	55
5	Frank	10	Male	20
6	Grace	85	Female	35
7	David	48	Male	52

□ Observation :

This code defines a function that scores job applicants based on education, experience, gender, and age. It then applies this function to a sample list of applicants and displays the resulting scores in a table. A simple observation is that the scores vary significantly among applicants based on the criteria used in the scoring function.

□ Explanation :

This code sets up a system to score job applicants. It has a function `score_applicant` that takes details like education, experience, gender, and age and calculates a score for each

person based on rules defined inside the function. It then uses this function to score a list of example applicants and shows the results in a table using a pandas DataFrame

- ☐ Task Description#5(Inclusiveness)

- Code Snippet

- ☐ Expected Output :

- Regenerate code that includes gender-neutral also.

- ☐ Prompt :

Modify the greeting function so it does not assume binary genders and can greet users in a gender-neutral way, while still respecting when someone wants to be greeted with Mr./Ms.

```
def greet(name, gender=None, title=None):  
    if title:  
        print(f"Hello, {title} {name}!")  
    elif gender and gender.lower() == 'female':  
        print(f"Hello, Ms. {name}!")  
    elif gender and gender.lower() == 'male':  
        print(f"Hello, Mr. {name}!")  
    else:  
        print(f"Hello, {name}!")  
greet("Alex")  
greet("Taylor", gender="Non-binary")  
greet("Jordan", title="Mx.")  
greet("Sarah", gender="Female")  
greet("David", gender="Male")  
greet("Dr. Lee", title="Dr.") |
```

```
⇒ Hello, Alex!  
Hello, Taylor!  
Hello, Mx. Jordan!  
Hello, Ms. Sarah!  
Hello, Mr. David!  
Hello, Dr. Dr. Lee!
```

- ☐ Observation :

This code defines a function called greet that prints a greeting message. A simple observation from the output is that the function can provide different greetings based on whether a title, gender, or neither is provided, including gender-neutral greetings when no specific gender or title is given. It also correctly uses the provided title when available.

- ☐ Explanation :

This code defines a function called `greet` that's used to say hello to people. It's designed to be flexible: If you provide a title (like "Mr.", "Ms.", or "Mx."), it will use that title in the greeting.

If you don't provide a title but specify a gender as "Female" or "Male", it will use "Ms." or "Mr." respectively.

If you don't provide a title or a recognizable gender, it just says "Hello," followed by the name, offering a gender-neutral greeting by default.